

name=value [*name=value*] ...

Pattern-matching is not performed on *value*.

\${parameter}

A *parameter* is a sequence of letters, digits, or underscores (a *name*), a digit, or any of the *, @, #, ?, -, \$, and ! characters. The value, if any, of the parameter is substituted. The braces are required only when *parameter* is followed by a letter, digit, or underscore that is not to be interpreted as part of its name. A *name* must begin with a letter or underscore. If *parameter* is a digit then it is a positional parameter. If *parameter* is * or @, then all the positional parameters, starting with \$1, are substituted (separated by spaces). Parameter \$0 is set from argument zero when the shell is invoked.

\${parameter:-word}

If *parameter* is set and is nonnull, substitute its value; otherwise substitute *word*.

\${parameter:=word}

If *parameter* is not set or is null, set it to *word*; the value of the parameter is then substituted. Positional parameters may not be assigned to in this way.

\${parameter:?word}

If *parameter* is set and is nonnull, substitute its value; otherwise, print *word* and exit from the shell. If *word* is omitted, the message "parameter null or not set" is printed.

\${parameter:+word}

If *parameter* is set and is nonnull, substitute *word*; otherwise, substitute nothing.

In the above examples, *word* is not evaluated unless it is to be used as the substituted string, so that in the following example, *pwd* is executed only if *d* is not set or is null:

```
echo ${d:-`pwd` }
```

If the colon (:) is omitted from the above expressions, the shell only checks whether *parameter* is set.

The following parameters are automatically set by the shell:

- # The number of positional parameters in decimal
- Flags supplied to the shell on invocation or by the *set* command
- ? The decimal value returned by the last synchronously executed command
- \$ The process number of this shell

! The process number of the last background command invoked

The following parameters are used by the shell:

HOME The default argument (home directory) for the *cd* command.

PATH The search path for commands (see *Execution*).

MAIL If this variable is set to the name of a mail file, then the shell informs the user of the arrival of mail in the specified file.

PS1 Primary prompt string, by default “\$”.

PS2 Secondary prompt string, by default “>”.

IFS Internal field separators, normally SPACEBAR, TAB, and NEWLINE.

The shell gives default values to PATH, PS1, PS2, and IFS, while HOME and MAIL are not set at all by the shell (although HOME is set by *login*(M)).

Blank Interpretation

After parameter and command substitution, the results of substitution are scanned for internal field separators (those found in IFS) and split into distinct arguments where these characters are found. Explicit null arguments (“” or ‘ ’) are retained. Implicit null arguments (those resulting from *parameters* that have no values) are removed.

Filename Generation

Following substitution, each command *word* is scanned for the *, ?, and [characters. If one of these characters appears, the word is regarded as a *pattern*. The word is replaced with alphabetically sorted filenames that match the pattern. If no filename is found that matches the pattern, the word is left unchanged. The . character at the start of a filename or immediately following a /, as well as the / character itself, must be matched explicitly. These characters and their matching patterns are:

* Matches any string, including the null string.

? Matches any single character.

[...]

Matches any one of the enclosed characters. A pair of characters separated by – matches any character lexically between the pair, inclusive. If the first character following the opening bracket ([]) is an exclamation mark (!) then any character not enclosed is matched.

Quoting

The following characters have a special meaning to the shell and cause termination of a word unless quoted:

; & () | ^ < > NEWLINE SPACEBAR TAB

A character may be *quoted* (i.e., made to stand for itself) by preceding it with a \. The pair \NEWLINE is ignored. All characters enclosed between a pair of single quotation marks (``''), except a single quotation mark, are quoted. Inside double quotation marks (""), parameter and command substitution occurs and \ quotes the \, ^, ", and \$ characters. "\$*" is equivalent to "\$1 \$2 ...", whereas "\$@" is equivalent to "\$1" "\$2"

Prompting

When used interactively, the shell prompts with the value of PS1 before reading a command. If at any time a NEWLINE is typed and further input is needed to complete a command, the secondary prompt is issued (i.e., the value of PS2).

Input/Output

Before a command is executed, its input and output may be redirected using a special notation interpreted by the shell. The following may appear anywhere in a simple-command or may precede or follow a *command*. They are *not* passed on to the invoked command; substitution occurs before *word* or *digit*; is used:

<word	Use file word as standard input (file descriptor 0).
>word	Use file word as standard output (file descriptor 1). If the file does not exist then it is created; otherwise, it is truncated to zero length.
>>word	Use file word as standard output. If the file exists then output is appended to it (by first seeking the end-of-file); otherwise, the file is created.
<<[-]word	The shell input is read up to a line that is the same as word , or to an end-of-file. The resulting document becomes the standard input. If any character of word is quoted, then no interpretation is placed upon the characters of the document; otherwise, parameter and command substitution occurs, (unescaped) NEWLINE is ignored, and a \ must be used to quote the \, \$, ^ characters, and the first character of word . If - is appended to <<, then all leading tabs are stripped from word and from the document.

- <&digit** The standard input is duplicated from file descriptor *digit* (see *dup(S)*). Similarly for the standard output using *>*.
- <&-** The standard input is closed. Similarly for the standard output using *>*.

If one of the above is preceded by a digit, then the file descriptor created is that specified by the digit (instead of the default 0 or 1). For example:

```
... 2>&1
```

creates file descriptor 2 that is a duplicate of file descriptor 1.

If a command is followed by *&*, the default standard input for the command is the empty file, */dev/null*. Otherwise, the environment for the execution of a command contains the file descriptors of the invoking shell as modified by input/output specifications.

Environment

The *environment* (see *environ(M)*) is a list of name-value pairs that is passed to an executed program in the same way as a normal argument list. The shell interacts with the environment in several ways. Upon invocation, the shell scans the environment and creates a parameter for each name found, giving it the corresponding value. Executed commands inherit the same environment. If the user modifies the values of these parameters or creates new ones, none of these affects the environment unless the *export* command is used to bind the shell's parameter to the environment. The environment seen by any executed command is thus composed of any unmodified name-value pairs originally inherited by the shell, plus any modifications or additions, all of which must be noted in *export* commands.

The environment for any *simple-command* may be augmented by prefixing it with one or more assignments to parameters. Thus:

```
TERM=450 cmd args
```

and

```
(export TERM; TERM=450; cmd args)
```

are equivalent (as far as the above execution of *cmd* is concerned).

If the *-k* flag is set, *all* keyword arguments are placed in the environment, even if they occur after the command name.

Signals

The INTERRUPT and QUIT signals for an invoked command are ignored if the command is followed by `&`; otherwise, signals have the values inherited by the shell from its parent, with the exception of signal 11 (see also the *trap* command below).

Execution

Each time a command is executed, the above substitutions are carried out. Except for the *Special Commands* listed below, a new process is created and an attempt is made to execute the command via *exec(S)*.

The shell parameter, PATH, defines the search path for the directory containing the command. Alternative directory names are separated by a colon (:). The default path is `:/bin:/usr/bin` (specifying the current directory, `/bin`, and `/usr/bin`, in that order). Note that the current directory is specified by a null pathname, which can appear immediately after the equal sign or between the colon delimiters anywhere else in the path list. If the command name contains a `/`, the search path is not used. Otherwise, each directory in the path is searched for an executable file. If the file has execute permission but is not an `a.out` file, it is assumed to be a file containing shell commands. A subshell (i.e., a separate process) is spawned to read it. A command in parentheses is also executed in a subshell.

Special Commands

The following commands are executed in the shell process and, except as specified, no input/output redirection is permitted for such commands:

: No effect; the command does nothing. A zero exit code is returned.

.file

Reads and executes commands from *file* and returns. The search path, PATH, is used to find the directory containing *file*.

break [*n*]

Exits from the enclosing *for* or *while* loop, if any. If *n* is specified then it breaks *n* levels.

continue [*n*]

Resumes the next iteration of the enclosing *for* or *while* loop. If *n* is specified then it resumes at the *n*th enclosing loop.

cd [*arg*]

Changes the current directory to *arg*. The shell parameter, HOME, is the default *arg*. The shell parameter, CDPATH, defines the search path for the directory containing *arg*. Alternative directory names are separated by a colon (:). The default path is `<null>` (specifying the current directory). Note that the current directory is specified by a null pathname, which can appear immediately after the equal sign or between the colon delimiters anywhere else in the path list. If *arg* begins with a `/`, the search path is not used. Otherwise, each directory in the

path is searched for *arg*.

eval [*arg* ...]

The arguments are read as input to the shell and the resulting command(s) executed.

exec [*arg* ...]

The command specified by shell without creating a new process. Input/output arguments may appear and, if no other arguments are given, may cause the shell input/output to be modified.

exit [*n*]

Causes a shell to exit with the exit status specified by *n*. If *n* is omitted then the exit status is that of the last command executed (an end-of-file will also cause the shell to exit.)

export [*name* ...]

The given *names* are marked for automatic export to the *environment* of subsequently executed commands. If no arguments are given, a list of all names that are exported in this shell is printed.

newgrp [*arg* ...]

Equivalent to *exec newgrp arg*

read [*name* ...]

One line is read from the standard input, and the first word is assigned to the first *name*, the second word to the second *name*, etc., with leftover words assigned to the last *name*. The return code is 0 unless an end-of-file is encountered.

readonly [*name* ...]

The given *names* are marked *readonly* and the values of these *names* may not be changed by subsequent assignment. If no arguments are given, a list of all *readonly* names is printed.

set [**-eknuvx** [*arg* ...]]

-e

If the shell is not interactive, exits immediately if a command exits with a nonzero exit status.

-k

Places all keyword arguments in the environment for a command, not just those that precede the command name.

-n

Reads commands but does not execute them.

-u

Treats unset variables as an error when substituting.

-v

Prints shell input lines as they are read.

-x

Prints commands and their arguments as they are executed. Although this flag is passed to subshells, it does not enable tracing in those subshells.

—

Does not change any of the flags; useful in setting \$1 to —.

Using + rather than — causes these flags to be turned off. These flags can also be used upon invocation of the shell. The current set of flags may be found in \$—. The remaining arguments are positional parameters and are assigned, in order, to \$1, \$2, If no arguments are given, the values of all names are printed.

shift

The positional parameters from \$2 . . . are renamed \$1

test

Evaluates conditional expressions. See *test(C)* for usage and description.

times

Prints the accumulated user and system times for processes run from the shell.

trap [*arg*] [*n*] . . .

Arg is a command to be read and executed when the shell receives signal(s) *n*. (Note that *arg* is scanned once when the trap is set and once when the trap is taken.) Trap commands are executed in order of signal number. The highest signal number allowed is 16. Any attempt to set a trap on a signal that was ignored upon entry to the current shell is ineffective. An attempt to trap on signal 11 (memory fault) produces an error. If *arg* is absent, all *n* trap(s) are reset to their original values. If *arg* is the null string, this signal is ignored by the shell and by the commands it invokes. If *n* is 0, the command *arg* is executed on exit from the shell. The *trap* command with no arguments prints a list of commands associated with each signal number.

ulimit [-f] [*n*]

Imposes a size limit of *n*. The default value as set in */etc/default/login* is 32767 blocks or approximately 16.5 megabytes. A non-root user can only lower the value of *ulimit*.

-f

Imposes a size limit of *n* blocks on files written by child processes (files of any size may be read). With no argument, the current limit is printed.

If no option is given, -f is assumed.

umask [*ooo*]

The user file-creation mask is set to the octal number *ooo* where *o* is an octal digit (see *umask(C)*). If *ooo* is omitted, the current value of the mask is printed.

wait

Waits for the specified process to terminate, and reports the termination status. If *n* is not given then all currently active child processes are waited for. The return code from this command is always 0.

Invocation

If the shell is invoked through *exec(S)* and the first character of argument 0 is `-`, commands are initially read from `/etc/profile` and then from `$HOME/.profile`, if such files exist. Thereafter, commands are read as described below, which is also the case when the shell is invoked as `/bin/sh`. The flags below are interpreted by the shell on invocation only; note that unless the `-c` or `-s` flag is specified, the first argument is assumed to be the name of a file containing commands, and the remaining arguments are passed as positional parameters to that command file:

- `-c string` If the `-c` flag is present, commands are read from *string*.
- `-s` If the `-s` flag is present, or if no arguments remain, commands are read from the standard input. Any remaining arguments specify the positional parameters. Shell output is written to file descriptor 2.
- `-t` If the `-t` flag is present, a single command is read and executed, and the shell exits. This flag is intended for use by C programs only and is not useful interactively.
- `-i` If the `-i` flag is present, or if the shell input and output are attached to a terminal, the shell is *interactive*. In this case, `TERMINATE` is ignored (so that *kill 0* does not kill an interactive shell) and `INTERRUPT` is caught and ignored (so that *wait* is interruptible). In all cases, `QUIT` is ignored by the shell.
- `-r` If the `-r` flag is present, the shell is a restricted shell (see *rsh(C)*).

The remaining flags and arguments are described under the *set* command.

Exit Status

Errors detected by the shell, such as syntax errors, cause the shell to return a nonzero exit status. If the shell is not being used interactively, execution of the shell file is abandoned. Otherwise, the shell returns the exit status of the last command executed (see also the *exit* command).

Files

`/etc/profile`

\$HOME/.profile

/tmp/sh*

/dev/null

See Also

a.out(F), cd(C), dup(S), env(C), environ(M), exec(S), fork(S), login(M), newgrp(C), pipe(S), profile(M), rsh(C), signal(S), test(C), umask(C), umask(S), wait(S)

Notes

The command *readonly* (without arguments) produces the same output as the command *export*.

If << is used to provide standard input to an asynchronous process invoked by &, the shell gets mixed up about naming the input document; a garbage file, */tmp/sh** is created, and the shell complains about not being able to find that file by another name.

Bugs

When running “stty -cs8”, a quoted string of characters will be interpreted with the high-bit set which displays an alternate character set. To avoid this, set “stty -cs7” to strip the high-bit.

Name

shutdown – Terminates all processing.

Syntax

```
/etc/shutdown [ time ] [ su ]
```

Description

Shutdown is part of the XENIX operation procedures. Its primary function is to terminate all currently running processes in an orderly and cautious manner. The *time* argument is the number of minutes before a shutdown will occur and can be in the range of 0 to 15. The default time is five minutes. The optional *su* argument lets the user go single-user, without completely shutting down the system. However, the system is shut down for multiuser use. *Shutdown* goes through the following steps. First, all users logged on the system are notified to log off the system by a broadcasted message. All file system super-blocks are updated before the system is stopped (see *sync(C)*). This must be done before rebooting the system, to ensure file system integrity.

See Also

sync(C), *umount(C)*, *wall(C)*

Diagnostics

The most common error diagnostic that will occur is *device busy*. This diagnostic appears when a particular file system could not be unmounted. See *umount(C)*.

Notes

Once *shutdown* has been invoked it must be allowed to run to completion and must *not* be interrupted by pressing BREAK or DELETE.



Name

skip – Skips volume(s) on the tape cartridge drive.

Syntax

skip number-of-volumes

Description

Skip skips the specified number of *volumes* on the tape that is in the drive.

A volume is defined as a set of data on the tape between closures of the device. For example, if you wanted to put two large files on the tape, each in a separate volume, you could say:

```
tdd if=/large1 of=/dev/rcmt0 bs=8  
tdd if=/large2 of=/dev/rmt0 bs=8
```

Volume one would contain the file /large1 and volume two would contain the file /large2.

See Also

mt(M), tdd(C), rewind(C)

Files

/dev/rcmt0



Name

sleep – Suspends execution for an interval.

Syntax

sleep *time*

Description

Sleep suspends execution for *time* seconds. It is used to execute a command after a certain amount of time as in:

```
(sleep 105; command)&
```

or to execute a command every so often, as in:

```
while true
do
    command
    sleep 37
done
```

See Also

alarm(S), sleep(S)

Notes

Time must be less than 65536 seconds.



Name

`sort` – Sorts and merges files.

Syntax

```
sort [-cmu] [-o output] [-y kmem] [-z recsz] [-dfiMnr] [-b] [-tx]
[+ pos1 [-pos2] ] [ files ]
```

Description

Sort sorts lines of all the named files together and writes the result on the standard output. The standard input is read if `-` is used as a filename or if no input files are named.

Comparisons are based on one or more sort keys extracted from each line of input. By default, there is one sort key, the entire input line, and ordering is lexicographic by bytes in machine collating sequence.

The following options alter the default behavior:

-c Checks that the input file is sorted according to the ordering rules; gives no output unless the file is out of sort.

-m Merges only, the input files are already sorted.

-u Unique: suppresses all but one in each set of lines having equal keys.

-o*output*

The argument given is the name of an output file to use instead of the standard output. This file may be the same as one of the inputs. There may be optional blanks between `-o` and *output*.

-y*kmem*

The amount of main memory used by the sort has a large impact on its performance. Sorting a small file in a large amount of memory is a waste. If this option is omitted, *sort* begins using a system default memory size, and continues to use more space as needed. If this option is presented with a value, *kmem*, *sort* will start using that number of kilobytes of memory, unless the administrative minimum or maximum is violated, in which case the corresponding extreme value will be used. Thus, `-y0` is guaranteed to start with minimum memory. By convention, `-y` (with no argument) starts with maximum memory.

-z*recsz*

The size of the longest line read is recorded in the sort phase so that buffers can be allocated during the merge phase. If the sort phase is omitted by the `-c` or `-m` options, a popular system default size will be used. Lines longer than the buffer size will cause *sort* to terminate abnormally. Supplying the actual number of bytes in the longest line to be merged (or some larger

value) will prevent abnormal termination.

The following options override the default ordering rules:

- d “Dictionary” order: only letters, digits and blanks (SPACEBARs and TABs) are significant in comparisons.
- f Fold lowercase letters into uppercase.
- i Ignore characters outside the ASCII range 040-0176 in non-numeric comparisons.
- M Compare as months. The first three non-blank characters of the field are folded to uppercase and compared so that “JAN” < “FEB” < ... < “DEC”. Invalid fields compare low to “JAN”. The -M option implies the -b option (see below).
- n An initial numeric string, consisting of optional blanks, optional minus sign, and zero or more digits with optional decimal point, is sorted by arithmetic value. The -n option implies the -b option (see below). Note that the -b option is only effective when restricted sort key specifications are in effect.
- r Reverse the sense of comparisons.

When ordering options appear before restricted sort key specifications, the requested ordering rules are applied globally to all sort keys. When attached to a specific sort key (described below), the specified ordering options override all global ordering options for that key.

The notation, *+pos1 -pos2*, restricts a sort key to one beginning at *pos1* and ending at *pos2*. The characters at positions *pos1* and *pos2* are included in the sort key (provided that *pos2* does not precede *pos1*). A missing *-pos2* means the end of the line.

Specifying *pos1* and *pos2* involves the notion of a field, a minimal sequence of characters followed by a field separator or a NEWLINE. By default, the first blank (SPACEBAR or TAB) of a sequence of blanks acts as the field separator. All blanks in a sequence of blanks are considered to be part of the next field; for example, all blanks at the beginning of a line are considered to be part of the first field. The treatment of field separators can be altered using the options:

- tx Use *x* as the field separator; *x* is not considered to be part of a field (although it may be included in a sort key). Each occurrence of *x* is significant (e.g., *xx* delimits an empty field).
- b Ignore leading blanks when determining the starting and ending positions of a restricted sort key. If the -b option is specified before the first *+pos1* argument, it will be applied to all *+pos1* arguments. Otherwise, the b flag may be attached independently to each *+pos1* or *-pos2* argument (see below).

Pos1 and *pos2* each have the form *m.n*, optionally followed by one or more of the **b**, **d**, **f**, **i**, **n**, or **r** flags. A starting position specified by *+m.n* is interpreted to mean the *n*+1st character in the *m*+1st field. A missing *.n* means *.0*, indicating the first character of the *m*+1st field. If the **b** flag is in effect, *n* is counted from the first non-blank in the *m*+1st field; *+m.0b* refers to the first non-blank character in the *m*+1st field.

A last position specified by *-m.n* is interpreted to mean the *n*th character (including separators) after the last character of the *m*th field. A missing *.n* means *.0*, indicating the last character of the *m*th field. If the **b** flag is in effect, *n* is counted from the last leading blank in the *m*+1st field; *-m.1b* refers to the first non-blank in the *m*+1st field.

When there are multiple sort keys, later keys are compared only after all earlier keys compare equally. Lines that compare equally are ordered with all significant bytes.

Examples

Sort the contents of **infile** with the second field as the sort key:

```
sort +1 -2 infile
```

Sort, in reverse order, the contents of **infile1** and **infile2**, placing the output in **outfile** and using the first character of the second field as the sort key:

```
sort -r -o outfile +1.0 -1.2 infile1 infile2
```

Sort, in reverse order, the contents of **infile1** and **infile2** using the first non-blank character of the second field as the sort key:

```
sort -r +1.0b -1.1b infile1 infile2
```

Print the password file sorted by the numeric user ID (the third colon-separated field):

```
sort -t: +2n -3 /etc/passwd
```

Print the lines of the already sorted file, **infile**, suppressing all but the first occurrence of lines having the same third field (the **-um** options with just one input file choose a unique representative from a set of equal lines.):

```
sort -um +2 -3 infile
```

Files

```
/usr/tmp/stm???
```

```
/etc/passwd
```

See Also

comm(C), join(C), uniq(C)

Diagnostics

Comments and exits with non-zero status for various trouble conditions (e.g., when input lines are too long), and for disorder discovered under the `-c` option. When the last line of an input file is missing a NEWLINE character, *sort* appends one, prints a warning message, and continues.

Name

`split` – Splits a file into pieces.

Syntax

```
split [ -n ] [file [ name ] ]
```

Description

Split reads *file* and writes it in *n*-line pieces (default 1000) onto a set of output files. The name of the first output file is *name* with *aa* appended, and so on lexicographically. If no output *name* is given, *x* is used by default.

If no input file is given, or if a dash (–) is given instead, the standard input file is used.

See Also

`bfs(C)`, `csplit(C)`



Name

`stty` – Sets the options for a terminal.

Syntax

`stty [ -a| -g| options ]`

Description

Stty sets certain terminal I/O options for the device that is the current standard input; without arguments, it reports the settings of certain options. With the `-a` option, *stty* reports all of the option settings; with the `-g` option, it reports current settings in a form that can be used as an argument to another *stty* command. The first two groups of options describe control modes for the system console and other display devices. Detailed information about the modes listed in the next five groups may be found in *tty(M)*. *Options* in the last group are implemented using *options* in the previous groups. The *options* are selected from the following:

Control Modes for the System Console

mono Selects the monochrome display as the output device for the system console. This mode is valid if an IBM-compatible monochrome adapter is present or if an IBM-compatible enhanced graphics adapter (EGA) is present and the EGA is currently in one of the monochrome display modes.

color Selects an IBM-compatible regular color display as the output device for the system console. This mode is valid if a color graphics adapter is present or if an IBM-compatible EGA is present and is currently in one of the color graphics compatibility modes.

enhanced Selects the enhanced color display as the output device for the system console. This mode is valid if an EGA is present and is currently in a non-monochrome display mode.

Control Modes for the Keyboard

KB_ENABLE

Fully enables the keyboard to transmit 7-bit ASCII character codes, 8-bit character codes, and extended character codes.

KB_DISABLE

Disables 8-bit and extended character code transmission from the key-

board.

Control Modes for the Attached Display Devices

B40x25 Selects 40x25 black and white text display mode.

C40x25 Selects 40x25 color text display mode.

B80x25 Selects 80x25 black and white text display mode.

C80x25 Selects 80x25 color display text mode.

BG320 Selects 320x200 black and white graphics display mode.

CG320 Selects 320x200 color graphics display mode.

BG640 Selects 640x200 black and white graphics display mode.

The seven options above are valid for the following configurations: IBM-compatible color graphics adapter attached to an IBM-compatible regular color display and IBM enhanced graphics adapter (Modes 0-6) attached to an IBM-compatible regular color display or an IBM-compatible enhanced color display.

MCAMODE

Reinitializes the monochrome graphics adapter.

Warning: If invalid options are used for any of these display modes, the behavior of the attached device(s) may be erratic.

Common Control Modes

parenb (-parenb)

Enables (disables) parity generation and detection.

parodd (-parodd)

Selects odd (even) parity.

cs5 cs6 cs7 cs8

Selects character size (see *tty(M)*).

0 Hangs up phone line immediately.

50 75 110 134 150 200 300 600

1200 1800 2400 4800 9600 exta

Sets terminal baud rate to the number given, if possible.

hupcl (-hupcl)

Hangs up (does not hang up) phone connection on last close.

hup (-hup)

Same as hupcl (-hupcl).

cstopb (-cstopb)

Uses two(one) stop bits per character.

cread (-cread)

Enables (disables) the receiver.

clocal (-clocal)

Assumes a line without (with) modem control.

*Input Modes***ignbrk (-ignbrk)**

Ignores (does not ignore) break on input.

brkint (-brkint)

Signals (does not signal) INTERRUPT on break.

ignpar (-ignpar)

Ignores (does not ignore) parity errors.

parmrk (-parmrk)

Marks (does not mark) parity errors (see *ty(M)*).

inpck (-inpck)

Enables (disables) input parity checking.

istrip (-istrip)

Strips (does not strip) input characters to 7 bits.

inlcr (-inlcr)

Maps (does not map) NL to CR on input.

igncr (-igncr)

Ignores (does not ignore) CR on input.

icrnl (-icrnl)

Maps (does not map) CR to NL on input.

iuclic (-iuclic)

Maps (does not map) uppercase alphabets to lowercase on input.

ixon (-ixon)

Enables (disables) START/STOP output control. Output is stopped by sending an ASCII DC3 and started by sending an ASCII DC1.

ixany (-ixany)

Allows any character (only DC1) to restart output.

ixoff (-ixoff)

Requests that the system send (not send) START/STOP characters when the input queue is nearly empty/full.

*Output Modes***opost (-opost)**

Post-processes output (does not post-process output; ignores all other output modes).

olcuc (-olcuc)

Maps (does not map) lowercase alphabets to uppercase on output.

onlcr (-onlcr)

Maps (does not map) NL to CR-NL on output.

ocrnl (-ocrnl)

Maps (does not map) CR to NL on output.

onocr (-onocr)

Does not (does) output CRs at column zero.

onlret (-onlret)

On the terminal NL performs (does not perform) the CR function.

ofill (-ofill)

Uses fill characters (use timing) for delays.

ofdel (-ofdel)

Fill characters are DELETes (NULs).

cr0 cr1 cr2 cr3

Selects style of delay for RETURNS (see *tty(M)*).

nl0 nl1

Selects style of delay for LINEFEEDs (see *tty(M)*).

tab0 tab1 tab2 tab3

Selects style of delay for horizontal TABs (see *tty(M)*).

bs0 bs1

Selects style of delay for BACKSPACES (see *tty(M)*).

ff0 ff1

Selects style of delay for FORMFEEDs (see *tty(M)*).

vt0 vt1

Selects style of delay for Vertical TABs (see *ty(M)*).

*Local Modes***isig (-isig)**

Enables (disables) the checking of characters against the special control characters INTERRUPT and QUIT.

icanon (-icanon)

Enables (disables) canonical input (ERASE and KILL processing).

xcase (-xcase)

Canonical (unprocessed) upper/lowercase presentation.

echo (-echo)

Echoes back (does not echo back) every character typed.

echoe (-echoe)

Echoes (does not echo) ERASE character as a SPACEBAR string. Note: this mode will erase the ERASE character on many CRT terminals; however, it does *not* keep track of column position and, as a result, may be confusing on escaped characters, TABs, and BACKSPACES.

echok (-echok)

Echoes (does not echo) NL after KILL character.

lfkc (-lfkc)

The same as **echok (-echok)**; obsolete.

echonl (-echonl)

Echoes (does not echo) NL.

noflsh (-noflsh)

Disables (enables) flush after INTERRUPT or QUIT.

stwrap (-stwrap)

Disables (enables) truncation of lines longer than 79 characters on a synchronous line.

stflush (-stflush)

Enables (disables) flush on a synchronous line after every *write(S)*.

stappl (-stappl)

Uses application (line) mode on a synchronous line.

*Control Assignments**control-character C*

Sets *control-character* to *C*, where *control-character* is erase, kill, intr, quit, eof, eol. If *C* is preceded by a caret (^) (escaped from the shell), then the value used is the corresponding CTRL character (e.g., “^D” is a CTRL-D); “^?” is interpreted as DELETE and “^_” is interpreted as undefined.

min i, time i (0<i<127)

When **-icanon** is not set, read requests are not satisfied until at least **min** characters have been received or the timeout value **time** has expired. See **tty(C)**.

line i

Sets the line discipline to *i* ($0 < i < 127$). There are currently no line disciplines implemented, so the legal value for *i* is 0.

*Combination Modes***evenp or parity**

Enables **parenb** and **cs7**.

oddp

Enables **parenb**, **cs7**, and **parodd**.

-parity, -evenp, or -oddp

Disables **parenb**, and sets **cs8**.

raw (-raw or cooked)

Enables (disables) raw input and output (no ERASE, KILL, INTERRUPT, QUIT, EOT, or output post-processing).

nl (-nl)

Unsets (sets) **icrnl**, **onlcr**. In addition **-nl** unsets **inlcr**, **igncr**, **ocrnl**, and **onlret**.

lcase (-lcase)

Sets (unsets) **xcase**, **iuclic**, and **olcuc**.

LCASE (-LCASE)

Same as **lcase (-lcase)**.

tabs (-tabs or tab3)

Preserves (expands to spaces) tabs when printing.

ek

Resets ERASE and KILL characters back to normal CTRL-H and CTRL-U.

sane

Resets all modes to some reasonable values. Useful when a terminal's settings have been hopelessly scrambled.

term

Sets all modes suitable for the terminal type, TERM, where TERM is one of **tty33**, **tty37**, **vt05**, **tn300**, **ti700**, or **tek**.

 See Also

`console(M)`, `ioctl(S)`, `tty(M)`

Notes

Many combinations of options make no sense, but no checking is performed.

Bugs

If you set your terminal to run at 19200 baud (exta), the *stty* command reports that your line is running at 0 baud.



Name

su – Makes the user a super-user or another user.

Syntax

```
su [-] [ name [ arg ... ] ]
```

Description

Su allows you to become another user without logging off. The default user *name* is **root** (i.e., super-user).

To use *su*, the appropriate password must be supplied (unless you are already a super-user). If the password is correct, *su* will execute a new shell with the real and effective user ID set to that of the specified user. The new shell will be the optional program named in the shell field of the specified user's password file (*/bin/sh* if none is specified (see *sh(C)*). To restore normal user ID privileges, type an EOF (CTRL-D) to the new shell.

Any additional arguments given on the command line are passed to the program invoked as the shell. When using programs like *sh(C)*, an *arg* of the form **-c string** executes *string* via the shell, and an *arg* of **-r** will give the user a restricted shell.

The following statements are true only if the optional program named in the shell field of the specified user's password file entry is like *sh(C)*. If the first argument to *su* is a **-**, the environment will be changed to what would be expected if the user actually logged in as the specified user. This is done by invoking the program used as the shell with an *arg0* value whose first character is **-**, thus causing first the system's profile (*/etc/profile*) and the specified user's profile (*.profile* in the new HOME directory) to be executed. Otherwise, the environment is passed along with the possible exception of *\$PATH*, which is set to */bin:/etc:/usr/bin* for **root**. Note that if the optional program used as the shell is */bin/sh*, the user's *.profile* can check *arg0* for **-sh** or **-su** to determine if it was invoked by *login(M)* or *su(C)*, respectively. If the user's program is other than */bin/sh*, then *profile* is invoked with an *arg0* of *-program* by both *login(M)* and *su(C)*.

If you wish to keep a record of all attempts to become another user using *su* in the log file, */usr/adm/sulog*, edit the file */usr/default/su* to remove the '#' at the start of the line:

```
#SU=/usr/adm/sulog
```

After this is done, each invocation of the *su* command puts a line in the */usr/adm/sulog* file of the form:

```
SU mm/dd hh:mm + tty user-newuser
```

where *mm/dd* is the month/day and *hh:mm* is the hour:minute of the use of the *su* command, *tty* is the terminal on which the command was entered, *user* is the user

id of the person entering the command, and *newuser* is the user id that the user attempted to become. This file should be checked periodically and edited if it becomes too large in order to prevent waste of disk space.

Examples

To become user **bin** while retaining your previously exported environment, type:

```
su bin
```

To become user **bin** but change the environment to what would be expected if **bin** had originally logged in, type:

```
su - bin
```

To execute *command* with the temporary environment and permissions of user **bin**, type:

```
su - bin -c "command args"
```

Files

/etc/passwd	The system password file
/etc/profile	The system profile
/etc/default/su	Default file to turn on log
\$HOME/.profile	The user profile
/usr/adm/sulog	The log file

See Also

default(M), env(C), environ(M), login(M), passwd(M), profile(M), sh(C)

Name

sulogin – Accesses single user mode.

Syntax

sulogin

Description

Sulogin is automatically invoked by *init* when the system is first started. It prompts the user to type the root password to enter system maintenance mode (single user mode) or to type CTRL-D for normal startup (multi-user mode). *Sulogin* should never be directly invoked by the user.

Files

/etc/sulogin

See Also

init(M)



Name

`sum` – Calculates checksum and counts blocks in a file.

Syntax

`sum [ -r ] file`

Description

Sum calculates and prints a 16-bit checksum for the named *file*, and also prints the number of blocks (512 bytes each) in the *file*. It is typically used to look for bad spots, or to validate a file communicated over a transmission line. The `-r` option causes an alternate algorithm to be used in computing the checksum.

See Also

`wc(C)`

Diagnostics

“Read error” is indistinguishable from end-of-file on most devices; check the block count.



Name

`sync` – Updates the super-block.

Syntax

`void sync( )`

Description

Sync executes the *sync* system primitive. If the system is to be stopped, *sync* must be called to ensure file system integrity. Note that *shutdown(C)* automatically calls *sync* before shutting down the system.

See Also

`sync(S)`



Name

`sysadmin` – Performs file system backups and restores files.

Syntax

`/etc/sysadmin`

Description

Sysadmin is a script for performing file system backups and for restoring files from backup media. It can do a daily incremental backup (level 9), or a periodic full backup (level 0). It can provide a listing of the files backed up and also has a facility to restore individual files from a backup.

You must be the super-user to use this program.

Files

`/tmp/backup.list`

See Also

`backup(C)`, `dumpdir(C)`, `mkfs(C)`, `restore(C)`



Name

tar – Archives files.

Syntax

tar [*key*] [*files*]

Description

Tar saves and restores files to and from an archive medium, which is typically a storage device such as a floppy disk, a tape, or a regular file. Its actions are controlled by the *key* argument. The *key* is a string of characters containing at most one function letter and possibly one or more function modifiers. Valid function letters are **c**, **t**, **x**, and **r**. Other arguments to the command are *files* (or directory names) specifying which files are to be backed up or restored. In all cases, appearance of a directory name refers to the files and (recursively) subdirectories of that directory.

The function portion of the *key* is specified by one of the following letters:

- r** The named *files* are written to the end of the archive. The **c** function implies this function.
- x** The named *files* are extracted from the archive. If a named file matches a directory whose contents have been written onto the archive, this directory is (recursively) extracted. The owner, modification time, and mode are restored (if possible). If no *files* argument is given, the contents of the archive are extracted. Note that if several files with the same name are on the archive, the last file overwrites all earlier ones.
- t** The names of the specified files are listed each time that they occur on the archive. If no *files* argument is given, all the names on the archive are listed.
- c** Creates a new archive; writing begins at the beginning of the archive, instead of after the last file. This command implies the **r** function.
- e** Prevents files from being split across volumes (tapes or floppy disks). If there is not enough room on the present volume for a given file, *tar* prompts for a new volume. This is only valid when the **-k** option is also specified on the command line.

The following characters may be used in addition to the letter that selects the desired function:

- 0,...,7 This modifier selects the drive on which the archive is mounted. The default drive is 1.
- v Normally, *tar* does its work silently. The v (verbose) option causes it to type the name of each file it treats, preceded by the function letter. With the t function, v gives more information about the archive entries than just the name.
- o Causes the extracted files to assume the owner and group ID of the user running the program rather than those on the archive tape.
- w Causes *tar* to print the action to be taken, followed by the name of the file, and then wait for the user's confirmation. If a word beginning with "y" is given, the action is performed. Any other input means "no."
- f Causes *tar* to use the next argument as the name of the archive instead of /dev/mt?. If the name of the file is a dash (-), *tar* writes to the standard output or reads from the standard input, whichever is appropriate. Thus, *tar* can be used as the head or tail of a pipeline. *Tar* can also be used to move hierarchies with the command:
- ```
cd fromdir; tar cf - . | (cd todir; tar xf -)
```
- b Causes *tar* to use the next argument as the blocking factor for archive records. The default is 1, the maximum is 20. The number of bytes in a block is BSIZE as defined in /usr/include/sys/param.h(1024).
- F Causes *tar* to use the next argument as the name of a file from which succeeding arguments are taken. The dash (-) is not a valid argument here.
- l Tells *tar* to print an error message if it cannot resolve all of the links to the files being backed up. If l is not specified, no error messages are printed.
- m Tells *tar* not to restore the modification times. The modification time of the file will be the time of extraction.
- k Causes *tar* to use the next argument as the size of an archive volume in kilobytes. The minimum value allowed is 250. This option is useful when the archive is not intended for a magnetic tape device, but for some fixed size device, such as floppy disk (See f above). Very large files are split into "extents" across volumes. When restoring from a multivolume archive, *tar* only prompts for a new volume if a split file has been partially restored.

- n** Indicates the archive device is not a magnetic tape. The **k** option implies this. Listing and extracting the contents of an archive are sped because *tar* can seek over files it wishes to skip. Sizes are printed in kilobytes instead of tape blocks.
- p** Indicates that files are extracted using their original permissions. It is possible that a non-super-user may be unable to extract files because of the permissions associated with the files or directories being extracted.

## Examples

If the name of a floppy disk device is `/dev/rfd1`, a *tar* format file can be created on this device by typing:

```
assign /dev/rfd
tar cvfbk /dev/rfd1 20 360 files
```

where *files* are the names of files you want archived, 20 is the blocking factor, and 360 is the capacity of the floppy disk in kilobytes. Note that arguments to key letters are given in the same order as the key letters themselves; thus, the **fbk** key letters have corresponding arguments, `/dev/rfd1`, 20, and 360. Note that if a *file* is a directory, the contents of the directory are archived recursively. To print a listing of the archive, type:

```
tar tvfb /dev/rfd1 20
```

At some later time you may want to extract the files from the floppy archive. You can do this by typing:

```
tar xvfb /dev/rfd1 20
```

The above command extracts all files from the archive using the same pathnames as those of the original archive. Because of this behavior, it is best to save archive files with relative pathnames rather than absolute ones, since directory permissions may not let you read the files into the specified absolute directories.

In the above examples, the **v** option is used to confirm the reading or writing of archive files on the screen. Also, a normal file could be substituted for the floppy device, `/dev/rfd1`, in the examples.

## Files

```
/etc/default/dumpdir Default devices
/tmp/tar*
```

## Diagnostics

Prints an error message about bad key characters and archive read/write errors.

Prints an error message if not enough memory is available to hold the link tables.

## Notes

Use raw devices (eg. `/dev/rfd0`) as opposed to block devices (`/dev/fd0`). This means that you must use the `b` option and specify an even number of blocks.

Never remove a floppy diskette from its drive until the drive light goes out.

There is no way to ask for the *n*th occurrence of a file.

The limit on filename length is 100 characters.

When archiving a directory that contains subdirectories, *tar* will only access those subdirectories that are within 17 levels of nesting. Subdirectories at higher levels will be ignored after *tar* displays an error message.

Don't do:

```
tar xff --
```

This would imply taking two things from the standard input at the same time.

## Name

tars – Archives files to tape.

## Syntax

**tars** [ *-option* ] [ *files* ]

## Description

*Tars* saves and restores files to and from magnetic tape using the *tar(C)* program. Its actions are controlled by the *option* argument. The *option* may be chosen from the following: **t**, **x**, **o**, **all**. Other arguments to the command are *files* (or directory names) specifying which files are to be backed up or restored. In all cases, appearance of a directory name refers to the files and (recursively) subdirectories of that directory.

The options function as follows:

- t** displays a list of the files on the tape and their owners, permissions, and other data.
- x** extracts the specified files from the tape.
- o** causes the extracted files to assume the owner and group ID of the user running the program rather than those on the archive tape. The *o* option is used with the *x* option and must be specified after the *-x*.

If neither *x* nor *t* are specified, then it is assumed that the user is creating an archive, and the specified files are archived to tape.

## See Also

mt(M), tar(C), tdd(C)



## Name

`tdd` – Buffers and copies a file.

## Syntax

`tdd [option=value] ...`

## Description

`tdd` copies the specified input file to the specified output. The standard input and output are used by default. The working buffer size may be specified to take better advantage of raw physical I/O.

`if=file`

Input filename; standard input is default

`of=file`

Output filename; standard output is default

`bs=n`

Sets both input and output block size,

`tdd` is especially suited to I/O on very fast physical devices because it allows reading and writing with huge buffer sizes.

## See Also

`mt(M)`, `rewind(C)`, `skip(C)`, `dd(C)`

## Diagnostics

*k+b bytes read(written)*

Number of Kbytes read(written)





## Name

`tee` – Creates a tee in a pipe.

## Syntax

```
tee [-i] [-a] [files] ...
```

## Description

*Tee* transcribes standard input to the standard output and makes copies in the *files*. The `-i` option ignores interrupts; the `-a` option causes the output to be appended to the *files* rather than overwriting them.

## Examples

The following example illustrates the creation of temporary files at each stage in a pipeline:

```
grep ABC | tee ABC.grep | sort | tee ABC.sort | more
```

This example shows how to *tee* output to the terminal screen:

```
grep ABC | tee /dev/tty | sort | uniq >final.file
```



## Name

test – Tests conditions.

## Syntax

test *expr*

[ *expr* ]

## Description

*Test* evaluates the expression *expr* and, if its value is true, returns a zero (true) exit status; otherwise, a *test* returns a nonzero exit status if there are no arguments. The following primitives are used to construct *expr*:

|                              |                                                                                                                           |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>-r</b> <i>file</i>        | True if <i>file</i> exists and is readable.                                                                               |
| <b>-w</b> <i>file</i>        | True if <i>file</i> exists and is writable.                                                                               |
| <b>-x</b> <i>file</i>        | True if <i>file</i> exists and is executable.                                                                             |
| <b>-f</b> <i>file</i>        | True if <i>file</i> exists and is a regular file.                                                                         |
| <b>-d</b> <i>file</i>        | True if <i>file</i> exists and is a directory.                                                                            |
| <b>-c</b> <i>file</i>        | True if <i>file</i> exists and is a character special file.                                                               |
| <b>-b</b> <i>file</i>        | True if <i>file</i> exists and is a block special file.                                                                   |
| <b>-u</b> <i>file</i>        | True if <i>file</i> exists and its set-user-ID bit is set.                                                                |
| <b>-g</b> <i>file</i>        | True if <i>file</i> exists and its set-group-ID bit is set.                                                               |
| <b>-k</b> <i>file</i>        | True if <i>file</i> exists and its sticky bit is set.                                                                     |
| <b>-s</b> <i>file</i>        | True if <i>file</i> exists and has a size greater than zero.                                                              |
| <b>-t</b> [ <i>fildevs</i> ] | True if the open file whose file descriptor number is <i>fildevs</i> (1 by default) is associated with a terminal device. |
| <b>-z</b> <i>s1</i>          | True if the length of <i>s1</i> string is zero.                                                                           |
| <b>-n</b> <i>s1</i>          | True if the length of the <i>s1</i> string is nonzero.                                                                    |
| <b>s1 = s2</b>               | True if the <i>s1</i> and <i>s2</i> strings are identical.                                                                |

|                         |                                                                                                                                                            |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>s1</i> != <i>s2</i>  | True if the <i>s1</i> and <i>s2</i> strings are <i>not</i> identical.                                                                                      |
| <i>s1</i>               | True if <i>s1</i> is <i>not</i> the null string.                                                                                                           |
| <i>n1</i> -eq <i>n2</i> | True if the integers, <i>n1</i> and <i>n2</i> , are algebraically equal. Any of the comparisons, -ne, -gt, -ge, -lt, and -le, may be used in place of -eq. |

These primaries may be combined with the following operators:

|                 |                                                       |
|-----------------|-------------------------------------------------------|
| !               | Unary negation operator                               |
| -a              | Binary AND operator                                   |
| -o              | Binary OR operator (-a has higher precedence than -o) |
| ( <i>expr</i> ) | Parentheses for grouping                              |

Notice that all the operators and flags are separate arguments to *test*. Notice also that parentheses are meaningful to the shell and, therefore, must be escaped.

### See Also

find(C), sh(C)

### Warning

In the second form of the command (i.e., the one that uses [], rather than the word *test*), the square brackets must be delimited by blanks.

## Name

`touch` – Updates access and modification times of a file.

## Syntax

`touch [ -amc ] [ mmddhhmm[yy] ] files`

## Description

*Touch* causes the access and modification times of each argument to be updated. If no time is specified (see *date*(C)) the current time is used. The `-a` and `-m` options cause *touch* to update only the access or modification times respectively (default is `-am`). The `-c` option silently prevents *touch* from creating the file if it did not previously exist.

The return code from *touch* is the number of files for which the times could not be successfully modified (including files that did not exist and were not created).

## See Also

`date`(C), `utime`(S)



## Name

`tr` – Translates characters.

## Syntax

```
tr [-cds] [string1 [string2]]
```

## Description

*Tr* copies the standard input to the standard output with substitution or deletion of selected characters. Input characters found in *string1* are mapped into the corresponding characters of *string2*. Any combination of the `-cds` options may be used:

- `-c`        Complements the set of characters in *string1* with respect to the universe of characters whose ASCII codes are 001 through 377 octal.
- `-d`        Deletes all input characters in *string1*.
- `-s`        Squeezes all strings of repeated output characters that are in *string2* to single characters.

The following abbreviation conventions may be used to introduce ranges of characters or repeated characters into the strings:

- `[a-z]`     Stands for the string of characters whose ASCII codes run from character *a* to character *z*, inclusive.
- `[a*n]`     Stands for *n* repetitions of *a*. If the first digit of *n* is 0, *n* is considered octal; otherwise, *n* is taken to be decimal. A zero or missing *n* is taken to be huge; this facility is useful for padding *string2*.

The escape character, `\`, may be used as in the shell to remove special meaning from any character in a string. In addition, a `\` followed by 1, 2, or 3 octal digits stands for the character whose ASCII code is given by those digits.

The following example creates a list of all the words in *file1* one per line in *file2*, where a word is taken to be a maximal string of alphabets. The strings are quoted to protect the special characters from interpretation by the shell; 012 is the ASCII code for NEWLINE:

```
tr -cs "[A-Z][a-z]" "\012*" <file1 >file2
```

**See Also**

ascii(M), ed(C), sh(C)

**Notes**

Won't handle ASCII NUL in *string1* or *string2*; always deletes NUL from input.



## Name

`true` – Returns with a zero exit value.

## Syntax

`true`

## Description

*True* does nothing except return with a zero exit value. *False*(C), *true*'s counterpart does nothing except return with a nonzero exit value. *True* is typically used in shell procedures such as:

```
while true
do
 command
done
```

## See Also

`false`(C), `sh`(C)

## Diagnostics

*True* has exit status zero.



## Name

**tset** – Provides information for setting terminal modes.

## Syntax

**tset** [ *options* ] [ *type* ]

## Description

*Tset* allows the user to set a terminal's ERASE and KILL characters, and define the terminal's type and capabilities by creating values for the TERM and TERMCAP environment variables. If a *type* is given with the **-s** option, *tset* creates information for a terminal of the specified type. The type may be any type given in */etc/termcap*. If the *type* is not specified with the **-s** option, *tset* creates information for a terminal of the type defined by the value of the environment variable, TERM, unless the **-h** or **-m** option is given. If the TERM variable is undefined, *tset* looks in */etc/termcap* for the appropriate information. If these options are used, *tset* searches the */etc/ttytype* file for the terminal type corresponding to the current serial port; it then creates information for a terminal based on this type. If the serial port is not found in */etc/ttytype*, the terminal type is set to **dumb**.

*Tset* displays the created information at the standard output. The information is in a form that can be used to set the current environment variables. The exact form depends on what the SHELL variable is set to in the environment. The examples below illustrate how to use this information to change the variables.

There are the following options:

- e**[*c*]      Sets the ERASE character to *c* on all terminals. The default setting is the BACKSPACE, or CTRL-H.
- E**[*c*]      Identical to the **-e** command except that it only operates on terminals that can BACKSPACE.
- k**[*c*]      Sets the KILL character to *c*, defaulting to CTRL-U.
- Prints the terminal type on the standard output.
- s**           Outputs the “setenv” commands (for *csh*(CP)), or “export” and assignment commands (for *sh*(C)). The type of commands are determined by what the SHELL variable is set to in the environment.
- h**           Forces *tset* to search */etc/ttytype* for information and to overlook the environment variable, TERM.

- S            Only outputs the strings to be placed in the environment variables.
- r            Prints the terminal type on the diagnostic output.
- Q            Suppresses the printing of the “Erase set to” and “Kill set to” messages.
- I            Suppresses printing of the terminal initialization strings.

–m[*ident*][*test baudrate*]:*type*

Allows a user to specify how a given serial port is to be mapped to an actual terminal type. The option applies to any serial port in */etc/ttytype* whose type is indeterminate (e.g., *dialup*, *plugboard*, etc.). The *type* specifies the terminal type to be used, and *ident* identifies the name of the indeterminate type to be matched. If no *ident* is given, all indeterminate types are matched. The *test baudrate* defines a test to be performed on the serial port before the type is assigned. The *baudrate* must be as defined in *stty*(C). The *test* may be any combination of: >, =, <, @, and !. If the *type* begins with a question mark, the user is asked if he really wants that type. A null response means to use that type; otherwise, another type can be entered which will be used instead. The question mark must be escaped to prevent filename expansion by the shell. If more than one –m option is given, the first correct mapping prevails.

*Tset* is most useful when included in the *.login* (for *csh*(CP)) or *.profile* (for *sh*(C)) file executed automatically at login, with –m mapping used to specify the terminal type you most frequently dial in on.

## Examples

```
tset gt42
```

```
tset -mdialup>300:adm3a -mdialup:dw2 -Qr -e#
```

```
tset -m dial:ti733 -m plug:\?hp2621 -m unknown:\? -e -k^U
```

To use the information created by the –s option for the Bourne shell, (*sh*), repeat these commands:

```
tset -s ... > /tmp/tset$$
/tmp/tset$$
rm /tmp/se:$$
```

To use the information create for *csh*, use:

```
set noglob
set term=('tset -S ...')
setenv TERM $term[1]
setenv TERMCAP "$term[2]"
unset term
```

unset noglob

## Files

/etc/ttytype      Port name to terminal type map database

/etc/termcap      Terminal capability database

## See Also

stty(C), termcap(M), tty(M)

## Notes

This utility was developed at the University of California at Berkeley and is used with permission.



## Name

`tty` – Gets the terminal's name.

## Syntax

`tty [ -s ]`

## Description

The `tty` command prints the pathname of the user's terminal on the standard output. The `-s` option inhibits printing, allowing you to test just the exit code.

## Exit Codes

0 if the standard input is a terminal, 1 otherwise.

## Diagnostics

*not a tty*      If the standard input is not a terminal and `-s` is not specified





## Name

`umask` – Sets file-creation mode mask.

## Syntax

`umask [ ooo ]`

## Description

The user file-creation mode mask is set to *ooo*. The three octal digits refer to read/write/execute permissions for *owner*, *group*, and *others*, respectively. Only the low-order 9 bits of *cmask* and the file mode creation mask are used. The value of each specified digit is “subtracted” from the corresponding “digit” specified by the system for the creation of any file (see *umask(S)* or *creat(S)*). This is actually a binary masking operation, thus the name “*umask*”. In general, binary ones remove a given permission and zeros have no effect at all. For example, *umask* 022 removes *group* and *others* write permission (files normally created with mode 777 become mode 755 ; files created with mode 666 become mode 644).

If *ooo* is omitted, the current value of the mask is printed.

*Umask* is recognized and executed by the shell. By default, login shells have a *umask* of 022.

## See Also

`chmod(C)`, `chmod(S)`, `creat(S)`, `sh(C)`, `umask(S)`



## Name

`umount` – Dismounts a file structure.

## Syntax

`/etc/umount special-device`

## Description

*Umount* announces to the system that the removable file structure previously mounted on the *special-device* device is to be removed. Any pending I/O for the file system is completed, and the file structure is flagged clean. For fuller explanation of the mount process, see *mount*(C).

## Files

`/etc/mnttab` Mount table

## See Also

`mnttab`(F), `mount`(C), `mount`(S)

## Diagnostics

*device busy*      An executing process is using a file on the named file system



## Name

`uname` – Prints name of the current XENIX system.

## Syntax

`uname [ -snrmuva ]`

## Description

*Uname* prints the current system name of the XENIX system on the standard output file. It is primarily used to determine which system you are using. The options cause selected information returned by *uname(S)* to be printed:

- `-s` Prints the system name (default).
- `-n` Prints the nodename (the nodename may be a name that the system is known by to a communications network).
- `-r` Prints the operating system release.
- `-m` Manufacturer prints original supplier of XENIX system.
- `-u` Prints software serial number.
- `-v` Prints the operating system version.
- `-a` Prints all of the above information.

## See Also

*uname(S)*



## Name

uniq – Reports repeated lines in a file.

## Syntax

```
uniq [-udc [+n] [-n]] [input [output]]
```

## Description

*Uniq* reads the *input* file and compares adjacent lines. In the normal case, the second and succeeding copies of repeated lines are removed; the remainder is written on the output file. *Input* and *output* should always be different. Note that repeated lines must be adjacent in order to be found; see *sort(C)*. If the **-u** flag is used, only the lines that are not repeated in the original file are output. The **-d** option specifies that one copy of these repeated lines is to be written. The normal mode output is the union of the **-u** and **-d** mode outputs.

The **-c** option supersedes the **-u** and **-d** options and generates an output report in default style with each line preceded by a count of the number of times it occurred.

The *n* arguments specify skipping an initial portion of each line in the comparison:

- n**        The first *n* fields together with any blanks before each are ignored. A field is defined as a string of characters without spaces and tabs separated by spaces and tabs from adjacent fields.
- +n**        The first *n* characters are ignored. Fields are skipped before characters.

## See Also

comm(C), sort(C)





**Name**

units – Converts units.

**Syntax**

units

**Description**

*Units* converts quantities expressed in various standard scales to their equivalents in other scales. It works interactively in this fashion:

You have: **inch**

You want: **cm**

\* 2.540000e+00

/ 3.937008e-01

A quantity is specified as a multiplicative combination of units preceded optionally by a numeric multiplier. Powers are indicated by positive integer suffixes; division is indicated by the standard sign:

You have: **15 lbs force/in2**

You want: **atm**

\* 1.020689e+00

/ 9.797299e-01

*Units* only does multiplicative scale changes; thus it can convert Kelvin to Rankine, but not Centigrade to Fahrenheit. Most familiar units, abbreviations, and metric prefixes are recognized, as well as the following:

**pi**      Ratio of circumference to diameter

**c**        Speed of light

**e**        Charge on an electron

**g**        Acceleration of gravity

**force**   Same as **g**

**mole**   Avogadro's number

**water**   Pressure head per unit height of water

**au**      Astronomical unit

**Pound** is not recognized as a unit of mass—**lb** is. Compound names are run together, (e.g. **lightyear**). British units that differ from their US counterparts are prefixed with “**br**”. For a complete list of units, type:

```
cat /usr/lib/unittab
```

## Files

```
/usr/lib/unittab
```

## Notes

When you execute the *units* command, a message appears asking you to wait while **/usr/lib/unittab** is loaded. The system then displays:

```
nn units; nnnn bytes
```

You have:

The first line of this message tells you how many different types of units are defined in the **/usr/lib/unittab** file and how many bytes the internal form of this occupies. The next line shows you the *units* command is in interactive mode and you can begin doing conversions.

## Name

**uuclean** – Clean-up the uucp spool directory.

## Syntax

**uuclean** [ options ] ...

## Description

*Uuclean* will scan the spool directory for files with the specified prefix and delete all those which are older than the specified number of hours.

The following options are available:

- ddirectory** Cleans *directory* instead of the spool directory.
- m** Sends mail to the owner of the file when it is deleted.
- ntime** Files whose age is more than *time* hours will be deleted if the prefix test is satisfied. Default time is 72 hours.
- ppre** Scans for files with *pre* as the file prefix. Up to 10 **-p** arguments may be specified. A **-p** without any *pre* following will cause all files older than the specified time to be deleted.
- x num** Provides debugging information. For greater values of *num*, more information is given.

This program will typically be started by *cron*(C).

## Files

|                              |                                                           |
|------------------------------|-----------------------------------------------------------|
| <code>/usr/lib/uucp</code>   | Directory with commands used by <i>uuclean</i> internally |
| <code>/usr/spool/uucp</code> | Spool directory                                           |

## See Also

**uucp**(C), **uux**(C).



## Name

**uucp**, **uulog**, **uuname** – Copies files from XENIX to XENIX.

## Syntax

**uucp** [ option ] ... source-file ... destination-file

**uulog** [ option ] ...

**uuname** [ -l ] [ -v ]

## Description

*Uucp* copies files named by the source-file arguments to the destination-file argument. A filename may be a pathname on your machine, or may have the form:

system-name!pathname

where *system-name* is taken from a list of system names which *uucp* knows about. Shell metacharacters *?\*[]* appearing in *pathname* will be expanded on the appropriate system.

Pathnames may be a full pathname, or a pathname preceded by *~user* where *user* is a user ID on the specified system and is replaced by that user's login directory. Anything else is prefixed by the current directory.

If the result is an erroneous pathname for the remote system, the copy will fail. If the destination file is a directory, the last part of the source filename is used.

*Uucp* preserves execute permissions across the transmission and gives 0666 read and write permissions (see *chmod(S)*).

The following options are interpreted by *uucp*:

- d**            Makes all necessary directories for the file copy.
- c**            Uses the source file when copying out rather than copying the file to the spool directory.
- m**            Sends mail to you when the copy is complete.
- n user**        Notifies *user* on the remote system that a file was sent.
- r**            Queues job but does not start the file transfer process. By default, a file transfer process is started each time *uucp* is invoked.

- `-s string` Prepends a *string* to *uucp* filenames so you can have an alternate directory structure for debugging purposes.
- `-x num` Provides debugging based on the value of *num*. For increasing values of *num*, more information is given.

*Uulog* maintains a summary log of *uucp* and *uux*(C) transactions in the file `/usr/spool/uucp/LOGFILE` by gathering information from partial log files named `/usr/spool/uucp/LOG.*.?`. *Uulog* removes the partial log files.

The options cause *uulog* to print log information:

- `-ssys`  
Prints information about work involving system *sys*.
- `-uuser`  
Prints information about work done for the specified *user*.

*Uuname* lists the *uucp* names of known systems. The `-l` option returns the local system name. The `-v` option will print additional information about each system. A description will be printed for each system that has a line of information in `/usr/lib/uucp/ADMIN`. The format of **ADMIN** is:

sysname tab description tab

## Files

- `/usr/spool/uucp` Spool directory
- `/usr/spool/uucppublic` Public directory for receiving and sending (PUBDIR)
- `/usr/lib/uucp/*` Other data and program files

## See Also

`uux`(C), `mail`(C)

## Notes

For security reasons, all files received by *uucp* should be owned by *uucp*.

The `-m` option will only work sending files or receiving a single file. Receiving multiple files specified by special shell characters `?*[ ]_` will not activate the `-m` option.

This version of *uucp* is based on a version 7 implementation of the program.

### Warning

The domain of remotely accessible files can (and for obvious security reasons, usually should) be severely restricted. You will very likely not be able to fetch files by pathname; ask a responsible person on the remote system to send them to you. For the same reasons you will probably not be able to send files to arbitrary pathnames. As distributed, the remotely accessible files are those whose names begin with `/usr/spool/uucppublic`.

### Bugs

If your system is called from another site and that site has a debug level of 7 or higher turned on, your system's debug will be turned on. This can cause unpredictable behavior on the remote site.





## Name

`uuninstall` – Administers uucp control files.

## Syntax

`/etc/uuninstall [-r ]`

## Description

The *uuninstall* program is used to manage the content of the control files used by the *uucp* communications system. It allows the user to change the contents of these files without using a text editor. The user need not know the detailed format of each of the control files, although one must be familiar with the function of the various fields within the files. These details are explained in the *XENIX Operations Guide*.

The *uuninstall* program can only be executed by the super-user. When invoked with the optional `-r` flag, *uuninstall* will not allow any of the files to be modified whether or not the user has made changes to the files.

If *uuninstall* finds any of the required *uucp* control files missing from the system, it will create them with the correct access permissions and ownership.

## Files

`/etc/systemid`  
`/usr/lib/uucp/USERFILE`  
`/usr/lib/uucp/L.sys`  
`/usr/lib/uucp/L-devices`  
`/usr/lib/uucp/L-dialcodes`

## See Also

`mkuser(C)`,

*XENIX Operations Guide, Appendix C*



## Name

`uustat` – Uucp status inquiry and job control.

## Syntax

`uustat [ option ] ...`

## Description

*Uustat* will display the status of, or cancel, previously specified *uucp* commands, or provide general status on *uucp* connections to other systems. The following options are recognized:

- `-mmch`**      Reports the status of accessibility of machine *mch*. If *mch* is specified as **`all`**, then the status of all machines known to the local *uucp* are provided.
- `-kjobn`**      Kills the *uucp* request whose job number is *jobn*. The killed *uucp* request must belong to the person issuing the *uustat* command unless he is the super-user.
- `-rjobn`**      Reactivates the job specified by *jobn*. That is, the specified job is touched so that its modification time is set to the current time. This prevents *uuclean* from deleting the job until the job's modification time reaches the limit imposed by *uuclean*.
- `-chour`**      Removes the status entries which are older than *hour* hours. This administrative option can only be initiated by the user **`uucp`** or the super-user.
- `-uuser`**      Reports the status of all *uucp* requests issued by *user*.
- `-ssys`**      Reports the status of all *uucp* requests which communicate with remote system *sys*.
- `-ohour`**      Reports the status of all *uucp* requests which are older than *hour* hours.
- `-yhour`**      Reports the status of all *uucp* requests which are younger than *hour* hours.
- `-jall`**      Reports the status of all the *uucp* requests.
- `-v`**      Reports the *uucp* status verbosely. If this option is not specified, a status code is printed with each *uucp* request.
- `-q`**      Lists the number of jobs and other control files queued for each machine. If a lock file exists for that system, its date of creation is listed.
- `-xnum`**      Provides debugging based on the value of *num*. For increasing values of *num*, more information is given.

When no options are given, *uustat* outputs the status of all *uucp* requests issued by the current user. Note that only one of the options **`-j`**, **`-m`**, **`-k`**, **`-c`** may be specified at a time.

For example, the command:

```
uustat -uhdc -smhtsa -y72 -v
```

prints the verbose status of all *uucp* requests that were issued by user *hdc* to communicate with system *mhtsa* within the last 72 hours. The job request status format is:

```
job-number user remote-system command-time status-time
```

where the *status* may be either an octal number or a verbose description.

The octal code corresponds to the following description:

| OCTAL | STATUS                                                |
|-------|-------------------------------------------------------|
| 00001 | The copy failed, but the reason cannot be determined. |
| 00002 | Permission to access local file is denied.            |
| 00004 | Permission to access remote file is denied.           |
| 00010 | Bad <i>uucp</i> command is generated.                 |
| 00020 | Remote system cannot create temporary file.           |
| 00040 | Cannot copy to remote directory.                      |
| 00100 | Cannot copy to local directory.                       |
| 00200 | Local system cannot create temporary file.            |
| 00400 | Cannot execute <i>uucp</i> .                          |
| 01000 | Copy succeeded.                                       |
| 02000 | Copy finished, job deleted.                           |
| 04000 | Job is queued.                                        |

The machine accessibility status format is:

```
system-name time status
```

where *time* is the latest status time and *status* is a self-explanatory description of the machine status.

## Files

|                      |                     |
|----------------------|---------------------|
| /usr/spool/uucp      | Spool directory     |
| /usr/lib/uucp/L_stat | System status file  |
| /usr/lib/uucp/R_stat | Request status file |

## See Also

[uucp\(C\)](#)